TECNOLÓGICO NACIONAL DE MÉXICO
Instituto Tecnológico de Culiacán

Tópicos de Inteligencia Artificial

Unidad 2

Tarea Bonus Modulo II

MAESTRO:

Zuriel Dathan Mora Félix

ALUMNOS:

Jesús Omar Langarica Ornelas
Jesús Miguel Velarde Arce

CARRERA:

Ingeniería en Sistemas Computacionales

REPOSITORIO: https://github.com/OmarLangarica/topicos-ia

Culiacán, Sinaloa, 10 de Octubre de 2026

Índice

1. Introducción 3
2. Desarrollo 4
2.1. Problema de Programación de Trabajos (JSSP) 4
2.1.1. Descripción 4
2.1.2. Representación 5
2.1.3. Solución propuesta: búsqueda tabú 7
2.2. Problema de las N Reinas 8
2.2.1. Descripción 8
2.2.2. Representación 9
2.2.3. Solución propuesta: recocido simulado 10
2.3. Árbol de expansión mínima (MST) 11
2.3.1. Descripción 11
2.3.2. Representación (definición de variables del sistema) 13
2.3.3. Solución propuesta: búsqueda tabú 14
2.4. Problema del agente viajero (TSP) 15
2.4.1. Descripción 15
2.4.2. Representación 16
2.4.3. Solución propuesta: recocido simulado 17
3. Conclusiones 18
4. Bibliografía 20

1. Introducción

Todos los días se toman decisiones en las que hay que escoger lo mejor entre muchas opciones: la ruta más corta para repartir paquetes, el orden en que una fábrica debe hacer sus piezas o la forma más barata de conectar varias oficinas a una misma red. A este tipo de decisiones se les llama problemas de optimización. Cuando las opciones se forman combinando elementos (ordenar ciudades, escoger conexiones, asignar tareas a máquinas) se habla de optimización combinatoria, y su principal dificultad es que el número de combinaciones crece de forma explosiva: aun una computadora muy rápida no alcanza a revisarlas todas cuando el problema es grande (Hernández Constantino, 2019; Osaba, 2020).

Este trabajo estudia cuatro problemas clásicos. El problema de programación de trabajos (JSSP) decide en qué orden usan las máquinas de un taller los trabajos pendientes para terminar lo antes posible; se ha estudiado desde la década de 1950 porque describe muy bien los procesos de producción (Hernández Constantino, 2019). El problema de las N reinas pide acomodar reinas en un tablero sin que se ataquen, y se usa como banco de pruebas para comparar métodos de búsqueda (Alba y Durán, 1997). El árbol de expansión mínima (MST) busca conectar todos los puntos de una red con el menor costo posible, algo necesario al diseñar redes de telecomunicaciones, de distribución eléctrica o de carreteras (Diaz Rodriguez et al., 2026). Por último, el problema del agente viajero (TSP) busca el recorrido más corto que pasa por todas las ciudades una sola vez, y está detrás de la planeación de rutas de reparto (Osaba, 2020).

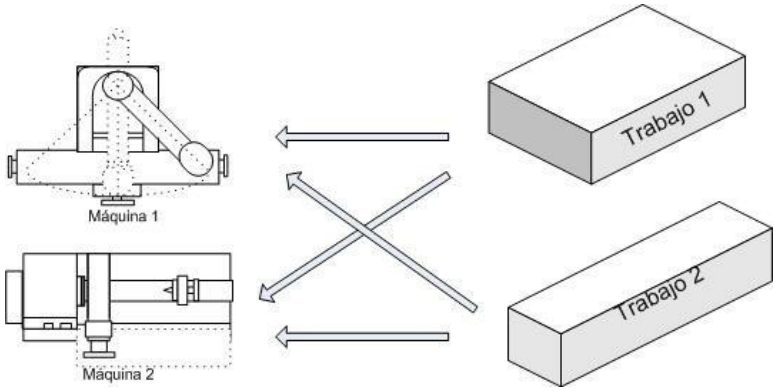
En las siguientes páginas cada problema se presenta en tres pasos. Primero se describe, con sus reglas y su objetivo. Después se muestra cómo se representa una solución. Finalmente se explica cómo se resolvería, con la metaheurística que mejor conviene (búsqueda tabú o recocido simulado) y las razones de esa elección.

2. Desarrollo

2.1. Problema de Programación de Trabajos (JSSP)

2.1.1. Descripción

El problema de planificación para producción tipo taller (JSSP, Job Shop Scheduling Problem) consiste en determinar la forma en que se deben utilizar un conjunto de recursos para realizar un conjunto de trabajos de forma que se minimice el tiempo requerido para procesar los mismos respetando a su vez un conjunto de restricciones, entre las que se incluyen restricciones relativas al orden en que se procesan las operaciones de los trabajos. (Hernández Constantino, 2019)



En el *Job Shop Scheduling Problem* hay una fábrica con varias máquinas y varios trabajos por hacer. Cada trabajo está formado por una lista ordenada de operaciones, y cada operación debe hacerse en una máquina concreta durante cierto tiempo. Por ejemplo, una pieza puede pasar primero por el torno, luego por la fresadora y al final por el taladro. Hay que decidir en qué orden cada máquina atiende las operaciones que le tocan. Las reglas son:

- Las operaciones de un mismo trabajo se hacen en su orden: no se puede empezar la segunda sin terminar la primera.
- Una máquina hace una sola operación a la vez.
- Una operación no se interrumpe una vez que empieza.

El objetivo es minimizar el makespan: el tiempo total desde que empieza el primer trabajo hasta que termina el último. Es un problema NP-difícil lo que significa que no se conoce un método rápido que siempre encuentre la mejor respuesta, y por eso se usan metaheurísticas. (Hernández Constantino, 2019)

Vélez, Castro y Maya (2003) describen el JSSP como un conjunto de *n* trabajos que deben procesarse en *m* máquinas, cada trabajo en un orden fijo que no puede violarse (la restricción de precedencia), y señalan que el objetivo más usado es minimizar el makespan. Como ejemplo típico ponen un taller de maquinado con tornos, fresadoras y taladros. Díaz Leyva y Medero Martínez (2020) estudian este mismo problema, llamado

en español *problema del taller mecánico*, en talleres de mecanizado con máquinas herramienta convencionales.

Por qué es difícil. El número de formas de ordenar los trabajos en las máquinas crece como $(n!)^m$, donde n es el número de trabajos y m el de máquinas: con 10 trabajos y 20 máquinas son unas 1.57×10^{131} posibilidades (Hernández Constantino, 2019). Además, no todas esas formas son válidas, como se verá en la representación.

2.1.2. Representación

Un taller con 3 trabajos y 3 máquinas. Cada operación se escribe como (máquina, duración):

Trabajo	Operación 1	Operación 2	Operación 3
J0	(M0, 3)	(M1, 2)	(M2, 2)
J1	(M0, 2)	(M2, 1)	(M1, 4)
J2	(M1, 4)	(M2, 3)	no tiene

La máquina M1 tiene que trabajar 2 + 4 + 4 = 10 unidades de tiempo en total, así que ningún cronograma puede terminar antes de 10.

Una solución es un cronograma completo. Se puede escribir de dos formas.

Forma 1: el orden en cada máquina. Para cada máquina se anota la lista de trabajos que atiende, en orden:

Máquina	Orden en que atiende los trabajos
M0	J0, J1
M1	J2, J0, J1
M2	J1, J2, J0

Es fácil de entender, pero tiene un problema: se pueden escribir órdenes **imposibles**. Por ejemplo, si M1 atiende a J1 antes que a J0, y M2 atiende a J0 antes que a J1, hay una contradicción. Siguiendo las reglas: J0 pasa por M1 antes que por M2; en M2, J0 va antes que J1; y J1 pasa por M2 antes que por M1. Entonces J0 en M1 tendría que ocurrir antes que J1 en M1, y eso contradice lo que se escribió para M1. Revisar que no haya contradicciones como ésta complica el trabajo. Hernández Constantino (2019) señala el mismo problema: no todas las ordenaciones por máquina son factibles, porque algunas forman ciclos que bloquean los trabajos.

Forma 2: lista de números de trabajo con repeticiones. Se escribe una lista con una posición por cada operación (8 en el ejemplo). En cada posición va el número de un trabajo, y el número de un trabajo aparece tantas veces como operaciones tiene (J0 tres veces, J1 tres veces, J2 dos veces). Se lee de izquierda a derecha: la primera vez que aparece un trabajo significa su operación 1, la segunda su operación 2, y así. Por ejemplo, la lista 0, 2, 1, 0, 1, 2, 1, 0 significa:

Posición	Número	Se lee como
1	0	operación 1 de J0
2	2	operación 1 de J2
3	1	operación 1 de J1
4	0	operación 2 de J0
5	1	operación 2 de J1
6	2	operación 2 de J2
7	1	operación 3 de J1
8	0	operación 3 de J0

La gran ventaja es que cualquier orden de la lista es válido: como la operación 2 de un trabajo siempre aparece después de su operación 1, se respeta la regla de orden y nunca aparecen contradicciones.

Cómo se calcula el cronograma. Se recorre la lista y cada operación empieza en el momento más temprano posible: cuando su trabajo ya terminó la operación anterior y su máquina ya está libre.

Paso	Operación	Máquina	Inicio	Fin	Por qué empieza ahí
1	J0, op. 1	M0	0	3	Todo está libre
2	J2, op. 1	M1	0	4	Todo está libre
3	J1, op. 1	M0	3	5	M0 está ocupada hasta 3
4	J0, op. 2	M1	4	6	M1 está ocupada hasta 4
5	J1, op. 2	M2	5	6	J1 termina su op. 1 en 5
6	J2, op. 2	M2	6	9	M2 está ocupada hasta 6
7	J1, op. 3	M1	6	10	M1 se libera en 6
8	J0, op. 3	M2	9	11	M2 se libera en 9

El mismo cronograma dibujado por máquina (cada columna es una unidad de tiempo; el número de la columna indica cuándo empieza esa unidad, y un punto significa que la máquina está libre):

Máquina	0	1	2	3	4	5	6	7	8	9	10
M0	J0	J0	J0	J1	J1	.	.	.	.	.	.
M1	J2	J2	J2	J2	J0	J0	J1	J1	J1	J1	.
M2	.	.	.	.	.	J1	J2	J2	J2	J0	J0

Función de costo: el makespan, que es el instante en que termina la última operación. En este cronograma es 11. Ese valor es el mejor posible para esta instancia: se comprobó revisando las 560 listas distintas que se pueden formar.

Cómo se puede modificar una solución. Si se intercambian los números de dos posiciones de la lista, se obtiene otra lista también válida. Por ejemplo, al intercambiar las posiciones 3 y 8 de 0, 2, 1, 0, 1, 2, 1, 0 queda 0, 2, 0, 0, 1, 2, 1, 1. Al calcular su cronograma, el makespan sube de 11 a 16. Así, cambios pequeños en la lista producen cronogramas distintos cuya calidad se puede comparar.

2.1.3. Solución propuesta: búsqueda tabú

Para resolver el JSSP se propone la búsqueda tabú. En la literatura sobre este problema es la metaheurística que mejores resultados ha dado: Hernández Constantino (2019) la eligió como base de su trabajo y explica que el recocido simulado, una de las primeras que se probaron, sólo logró resolver las instancias más fáciles. Además, encaja bien con la representación elegida: en el JSSP muchos cambios dejan el makespan igual, y la memoria de la lista tabú evita que la búsqueda dé vueltas entre soluciones equivalentes.

Cómo funciona, paso a paso:

- **Solución inicial:** una lista de números de trabajo mezclada al azar, por ejemplo 2, 0, 1, 1, 0, 2, 0, 1. Siempre es válida.
- **Costo:** se calcula el cronograma de la lista y se mide el makespan.
- **Vecindario:** intercambiar dos posiciones de la lista. Con 8 posiciones hay 28 intercambios posibles, es decir, 28 vecinos.
- **En cada iteración:** se calcula el makespan de los 28 vecinos y se elige el mejor que no esté prohibido, aunque sea peor que la solución actual.
- **Lista tabú:** se anota el par de posiciones intercambiado y se prohíbe repetirlo durante las siguientes 7 iteraciones. Así el algoritmo no deshace su último paso.
- **Aspiración:** un intercambio prohibido se permite si produce un makespan mejor que el mejor encontrado hasta ese momento.
- **Fin:** se guarda siempre la mejor lista vista y se termina después de un número fijo de iteraciones (por ejemplo, 100).

Ejemplo. Si la solución actual es 0, 2, 1, 0, 1, 2, 1, 0 (makespan 11), intercambiar las posiciones 3 y 8 da 0, 2, 0, 0, 1, 2, 1, 1 (makespan 16). La búsqueda tabú calcula así los 28 vecinos y se mueve al de menor makespan. Si ninguno mejora, de todos modos se mueve al menos malo, y ese intercambio queda prohibido por 7 iteraciones.

Valores de los parámetros. La tenencia tabú (cuántas iteraciones dura una prohibición) no debe ser muy corta, porque el algoritmo podría ciclarse, ni muy larga, porque prohibiría demasiados movimientos (Hernández Constantino, 2019). Para este tamaño se sugiere una tenencia de 7 y 100 iteraciones. En instancias grandes, revisar todos los vecinos es

el paso más costoso, por lo que suelen usarse vecindades reducidas (Hernández Constantino, 2019).

2.2. Problema de las N Reinas

2.2.1. Descripción

En el ajedrez, una reina ataca a cualquier pieza que esté en su misma fila, su misma columna o diagonal. El problema pide colocar N reinas en un tablero de N × N casillas de modo que ninguna ataque a otra. Es decir, no puede haber dos reinas en la misma fila, ni en la misma columna, ni en la misma diagonal.

Reglas del problema:

- El tablero tiene N filas y N columnas, y se colocan exactamente N reinas.
- Dos reinas no pueden compartir fila.
- Dos reinas no pueden compartir columna.
- Dos reinas no pueden compartir diagonal (en ninguno de los dos sentidos).

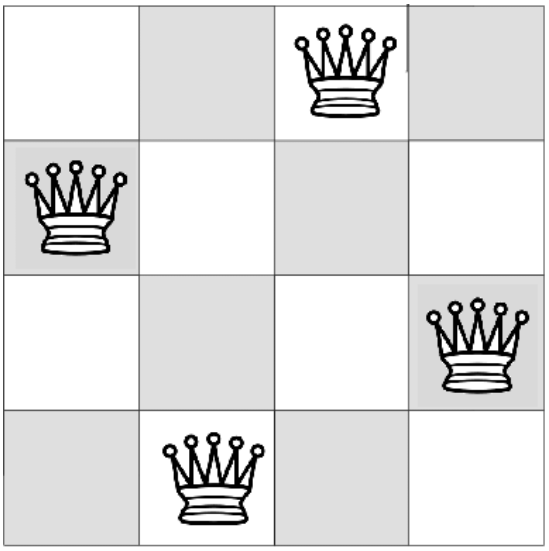
Objetivo: encontrar una colocación sin ataques. Aquí no hay soluciones "mejores" o "peores": una colocación es correcta o no lo es. Para tratarlo como un problema de optimización se cuenta cuántos pares de reinas se atacan, y la meta es llegar a cero.

Datos del problema: Para N = 2 y N = 3 no existe solución; desde N = 4 siempre existe al menos una. Con N = 8 hay 92 soluciones. El problema se generaliza a cualquier tamaño de tablero, y por eso se usa para comparar métodos.

Osaba (2020) explica que el problema de las N reinas generaliza el clásico problema de colocar 8 reinas en un tablero de ajedrez, propuesto por Max Bezzel en 1848, y que se puede plantear como un problema de optimización: la solución se escribe como una permutación y el objetivo es minimizar las colisiones en diagonal. Con esa representación se evitan las colisiones de fila y de columna, que es la misma idea de la lista de columnas que se usa en este documento. Alba y Durán (1997) lo emplean como banco de pruebas para comparar métodos de búsqueda, porque con él se pueden crear instancias de dificultad creciente (en su estudio, hasta N = 1024 reinas).

2.2.2. Representación

Forma 1: Se dibuja el tablero y se marca dónde hay una reina. Este tablero de 4 × 4 es una solución, porque ninguna reina ataca a otra:



Pero guardar el tablero completo ocupa mucho espacio: colocar 8 reinas en cualquier casilla de un tablero de 64 casillas da 4 426 165 368 posibilidades, y casi todas tienen reinas atacándose en filas o columnas.

Forma 2: Se escribe una lista con N números. La posición en la lista indica la fila y el número indica la columna de la reina de esa fila. El tablero de arriba se escribe 2, 0, 3, 1:

Fila	0	1	2	3
Columna de su reina	2	0	3	1

Esta forma es inteligente por dos razones:

- Hay exactamente una reina por fila, porque la lista tiene un número por fila.
- Si los números de la lista son todos distintos (una permutación de 0 a N−1), no hay dos reinas en la misma columna.

Entonces los ataques por fila y por columna son imposibles, y sólo queda revisar las diagonales. Además, el espacio de búsqueda baja mucho: para N = 8 sólo hay 8! = 40 320 listas, en lugar de más de 4 mil millones de tableros.

Función de costo: cuántos pares de reinas se atacan. Dos reinas en las filas *i* y *j* están en la misma diagonal cuando la diferencia entre sus columnas (sin signo) es igual a la diferencia entre sus filas. Ejemplo con la lista 1, 0, 2, 3, cuyo tablero es:

	Col 0	Col 1	Col 2	Col 3
Fila 0	·	♛	·	·
Fila 1	♛	·	·	·
Fila 2	·	·	♛	·
Fila 3	·	·	·	♛

Se revisan los 6 pares de filas:

Par de filas	Diferencia de filas	Diferencia de columnas	¿Se atacan?
0 y 1	1	1 (columnas 1 y 0)	**Sí**
0 y 2	2	1 (columnas 1 y 2)	No
0 y 3	3	2 (columnas 1 y 3)	No
1 y 2	1	2 (columnas 0 y 2)	No
1 y 3	2	3 (columnas 0 y 3)	No
2 y 3	1	1 (columnas 2 y 3)	**Sí**

Hay 2 pares que se atacan (las reinas de las filas 0 y 1, y las de las filas 2 y 3), así que el costo de la lista 1, 0, 2, 3 es **2**. La lista 2, 0, 3, 1 del tablero de arriba tiene costo **0**, por eso es una solución. Para N = 8, una solución es 0, 4, 7, 5, 2, 6, 1, 3.

Cómo se puede modificar una solución. Si se intercambian los números de dos posiciones de la lista, se mueven dos reinas y la lista sigue siendo una permutación (sin columnas repetidas). Por ejemplo, 0, 1, 2, 3 (todas en la diagonal, costo 6) pasa a 1, 0, 2, 3 (costo 2) al intercambiar las filas 0 y 1. Al intercambiar las filas 0 y 2 se llega a 2, 0, 1, 3 (costo 1), y al intercambiar las filas 2 y 3 se llega a 2, 0, 3, 1 (costo 0).

2.2.3. Solución propuesta: recocido simulado

Para las N reinas se propone el recocido simulado. Este método imita el enfriamiento lento de un metal: parte de una solución y prueba cambios pequeños al azar; si el cambio mejora, lo acepta siempre, y si empeora, lo acepta sólo a veces, con una probabilidad que baja a medida que baja la "temperatura" (Hernández Constantino, 2019). Se eligió por estas razones:

- **El problema puede crecer mucho.** Con N reinas hay N(N − 1)/2 intercambios posibles: 28 para N = 8, pero más de 500 mil para N = 1024. Revisar todos los vecinos en cada paso, como hace la búsqueda tabú, se vuelve muy caro, mientras que el recocido simulado prueba un solo vecino por paso.

- **Hay evidencia a favor.** Alba y Durán (1997) compararon varios métodos de búsqueda en este problema, con instancias de hasta N = 1024 reinas, y encontraron que el recocido simulado es muy eficiente en tiempo de ejecución.
- **Aceptar empeoramientos ayuda a salir de atascos.** Una permutación puede quedar con pocos ataques y ningún intercambio la mejora; aceptar de vez en cuando un cambio peor permite salir de ahí.

Cómo funciona, paso a paso:

- **Solución inicial:** una permutación de columnas al azar.
- **Costo:** número de pares de reinas que se atacan en diagonal. La meta es 0.
- **Vecino:** se eligen dos filas al azar y se intercambian sus columnas.
- **Aceptación:** si el costo baja o se queda igual, se acepta. Si sube en Δ, se acepta con probabilidad $e^{-\Delta/T}$, donde T es la temperatura.
- **Enfriamiento:** después de varios intentos con la misma temperatura, T se multiplica por un factor menor que 1 (por ejemplo 0.95).
- **Fin:** cuando el costo llega a 0 (se encontró una solución) o cuando T llega a un valor mínimo.

Ejemplo. Estando en la lista 1, 0, 2, 3 (costo 2), el algoritmo prueba dos cambios al azar:

Cambio probado	Lista resultante	Costo	Decisión
Intercambiar filas 0 y 2	2, 0, 1, 3	1	Baja de 2 a 1 (Δ = −1): se acepta siempre
Intercambiar filas 2 y 3	1, 0, 3, 2	4	Sube de 2 a 4 (Δ = +2): con T = 2 se acepta con probabilidad 0.37; con T = 0.2 casi nunca (0.00005)

Valores de los parámetros. Como los costos cambian en uno o dos ataques, basta una temperatura inicial pequeña. Para N = 8 se sugiere T inicial = 2, factor de enfriamiento 0.95, temperatura mínima 0.05 y 20 intentos por temperatura. Con T = 2, un empeoramiento de 1 ataque se acepta con probabilidad 0.61, así que al principio se explora bastante. La principal dificultad del método es escoger cómo baja la temperatura (Hernández Constantino, 2019).

2.3. Árbol de expansión mínima (MST)

2.3.1. Descripción

Un grafo es un conjunto de puntos, llamados vértices, unidos por líneas, llamadas aristas. En un grafo ponderado cada arista tiene un número, su peso, que puede ser un costo, una distancia o un tiempo. Por ejemplo, los vértices pueden ser edificios, las aristas los cables que se pueden tender entre dos edificios, y el peso el costo de cada cable.

Supón que hay que conectar todos los edificios a la misma red gastando lo menos posible. Basta con elegir algunas aristas de modo que:

- **Todos los vértices queden conectados:** desde cualquiera se puede llegar a cualquier otro siguiendo las aristas elegidas.
- **No haya ciclos:** no hay un camino que salga de un vértice y regrese a él sin repetir aristas. Un ciclo sería un cable de sobra, porque quitándole una arista el grafo seguiría conectado.

Un conjunto de aristas con esas dos propiedades se llama árbol de expansión, y siempre tiene V − 1 aristas (V es el número de vértices). El problema pide el árbol de expansión de peso total mínimo: el *Minimum Spanning Tree* (MST).

Reglas del problema:

- El grafo es no dirigido (las aristas se pueden recorrer en ambos sentidos), conexo y con pesos.
- Sólo se pueden elegir aristas que existan en el grafo.
- La selección debe conectar todos los vértices y no formar ciclos.

Objetivo: que la suma de los pesos de las aristas elegidas sea la menor posible.

En un grafo en el que todos los vértices están unidos entre sí, con n vértices existen n^{n-2} árboles de expansión; con 10 vértices son 100 millones. Aun así, el MST es un problema fácil: los algoritmos de Kruskal y de Prim lo resuelven rápido y siempre dan la mejor respuesta. Por eso sirve para practicar la representación y para comprobar resultados.

En español, a los árboles de expansión mínima también se les llama *árboles cobertores mínimos*; Ortega y Ramos (2012) analizan la eficiencia de cuatro variantes del algoritmo de Kruskal para construirlos. Diaz Rodriguez et al. (2026) explican el algoritmo de Prim: es un algoritmo voraz que parte de un vértice cualquiera y agrega, paso a paso, la arista de menor peso que une un vértice del árbol con uno que todavía no está en él. Señalan aplicaciones en redes de telecomunicaciones, distribución eléctrica, carreteras y cableado de redes informáticas, que son casos de conectar todo con el menor costo.

2.3.2. Representación (definición de variables del sistema)

Un grafo con 6 vértices (numerados de 0 a 5) y 9 aristas. Se muestra como lista de cada vértice con sus vecinos y el peso de cada conexión entre paréntesis:

Vértice	Está conectado con (peso)
0	1 (4), 2 (3)
1	0 (4), 2 (1), 3 (2)
2	0 (3), 1 (1), 3 (4), 4 (5)
3	1 (2), 2 (4), 4 (2), 5 (7)
4	2 (5), 3 (2), 5 (6)
5	3 (7), 4 (6)

Las variables del sistema son las siguientes:

Variable	Qué representa	En el ejemplo
V	Número de vértices	6
E	Conjunto de aristas disponibles	9 aristas
w(e)	Peso de la arista e	Valores de 1 a 7
T	Solución: selección de V − 1 aristas	Una lista de 5 números
C(T)	Costo de la solución	Suma de pesos más penalización

Paso 1: Cada arista recibe un número de identificación (ID) para poder nombrarla:

ID	0	1	2	3	4	5	6	7	8
Arista	0–1	0–2	1–2	1–3	2–3	3–4	4–5	3–5	2–4
Peso	4	3	1	2	4	2	6	7	5

Paso 2: Como un árbol de 6 vértices tiene 5 aristas, la solución es una lista de 5 IDs distintos. Por ejemplo, 1, 2, 3, 5, 6 significa que se eligen las aristas 0–2, 1–2, 1–3, 3–4 y 4–5. Otra forma equivalente es un vector con un 1 donde la arista está elegida y un 0 donde no: 0, 1, 1, 1, 0, 1, 1, 0, 0. Hay 126 maneras de escoger 5 aristas de 9.

Paso 3: revisar si la selección es un árbol**.** Una selección de 5 aristas es un árbol si conecta los 6 vértices: se parte de un vértice, se camina por las aristas elegidas y se cuenta a cuántos vértices se llega. Si se llega a los 6, está conectada, y con 5 aristas no puede haber ciclos. Si algún vértice queda fuera, hay vértices desconectados y la selección no es válida.

Función de costo. Se suman los pesos de las aristas elegidas y, para castigar las selecciones inválidas, se suman 10 por cada grupo de vértices que quede separado del

resto. El 10 es mayor que el peso de cualquier arista, para que dejar un vértice sin conectar nunca resulte más barato que conectarlo.

Ejemplo con tres selecciones:

Selección (IDs)	Aristas	Suma de pesos	¿Conectada?	Costo
1, 2, 3, 5, 6	0–2, 1–2, 1–3, 3–4, 4–5	3 + 1 + 2 + 2 + 6 = 14	Sí	**14**
1, 2, 3, 5, 7	0–2, 1–2, 1–3, 3–4, 3–5	3 + 1 + 2 + 2 + 7 = 15	Sí	15
2, 3, 4, 5, 6	1–2, 1–3, 2–3, 3–4, 4–5	1 + 2 + 4 + 2 + 6 = 15	No: el vértice 0 queda solo y 1–2–3 forma un ciclo	15 + 10 = 25

La tercera selección muestra para qué sirve la penalización: es una solución inválida y su costo se dispara.

La respuesta exacta (Kruskal). Se ordenan las aristas de menor a mayor peso y se aceptan una por una, saltando las que cerrarían un ciclo. Se aceptan 1–2 (1), 1–3 (2), 3–4 (2), 0–2 (3) y 4–5 (6); se rechazan 0–1, 2–3, 2–4 y 3–5 porque formarían ciclos. El árbol mínimo pesa 14, que es la primera selección de la tabla.

Cómo se puede modificar una solución. Se intercambia una arista: se quita una de las 5 aristas elegidas y se pone en su lugar una de las 4 que no estaban. Hay 5 × 4 = 20 cambios posibles. En la tabla anterior, la segunda selección sale de la primera quitando la arista 4–5 y poniendo la 3–5.

2.3.3. Solución propuesta: búsqueda tabú

El MST ya se resuelve de forma exacta y rápida con Kruskal o Prim (Ortega y Ramos, 2012; Diaz Rodriguez et al., 2026). Pero si se quisiera resolver con una metaheurística, la más conveniente sería la búsqueda tabú. Esta metaheurística le da memoria a la búsqueda: se mueve al mejor vecino aunque sea peor que la solución actual, y anota los últimos cambios en una lista de movimientos prohibidos para no regresar a donde ya estuvo (Hernández Constantino, 2019). Se eligió por estas razones:

- **El vecindario es pequeño:** sólo hay 5 × 4 = 20 cambios posibles, así que revisar todos en cada iteración es barato, y esa revisión completa es justo lo que hace la búsqueda tabú.
- **El cambio es fácil de recordar:** basta anotar qué arista salió y cuál entró. Prohibir ese par durante unas iteraciones evita deshacer el cambio de inmediato.
- **La penalización crea "valles":** las selecciones inválidas cuestan más, y la búsqueda tabú puede cruzarlos porque siempre avanza al mejor vecino permitido, aunque sea peor.

Cómo funciona, paso a paso:

- **Solución inicial:** 5 aristas elegidas al azar.
- **Costo:** suma de pesos más 10 por cada grupo de vértices desconectado.
- **Cada iteración:** se calcula el costo de los 20 cambios posibles y se elige el mejor que no esté prohibido.
- **Lista tabú:** se anota el cambio hecho (qué posición cambió y qué arista entró) y se prohíbe repetirlo durante 5 iteraciones.
- **Aspiración:** se permite un cambio prohibido si da un costo mejor que el mejor encontrado.
- **Fin:** después de un número fijo de iteraciones (por ejemplo, 100), se entrega la mejor selección vista.

Ejemplo. Si se parte de la selección 2, 3, 4, 5, 6 (costo 25, porque el vértice 0 queda solo), el mejor de los 20 vecinos se obtiene quitando la arista 2–3 (ID 4) y poniendo la 0–2 (ID 1). Resulta la selección 1, 2, 3, 5, 6, que conecta todo y pesa 14, el mismo valor que da Kruskal. Como 14 es el mínimo posible, en las siguientes iteraciones el algoritmo se mueve a vecinos de costo 15 (por ejemplo, cambiar 4–5 por 3–5), pero guarda siempre la mejor selección encontrada. Que Kruskal dé el óptimo permite comprobar si la metaheurística llegó a la mejor respuesta.

2.4. Problema del agente viajero (TSP)

2.4.1. Descripción

Un repartidor debe visitar *n* ciudades. Sabe la distancia entre cada par de ciudades y quiere hacer un recorrido que pase por todas exactamente una vez y regrese a la ciudad de donde salió, recorriendo la menor distancia total posible. Es el *Traveling Salesman Problem* (TSP), o problema del agente viajero. Aparece en la entrega de paquetes, en la perforación de placas de circuitos y en la planeación de rutas.

Reglas del problema:

- Se debe visitar cada ciudad exactamente una vez: ninguna se omite y ninguna se repite.
- El recorrido termina en la ciudad de inicio (es un ciclo cerrado).
- Se conoce la distancia entre cualquier par de ciudades.

Objetivo: que la suma de las distancias del recorrido completo, incluyendo el regreso, sea la menor posible.

Por qué es difícil. Con n ciudades hay (n − 1)! / 2 recorridos distintos cuando la distancia de ida y vuelta es la misma:

Ciudades	Recorridos posibles
4	3
8	2 520
10	181 440
20	unos 6×10^{16}
30	unos 4×10^{30}

Osaba (2020) define el TSP con un conjunto de nodos y una matriz de distancias entre ellos, y explica que probar todas las rutas (fuerza bruta) sólo es razonable con pocas ciudades, porque el número de permutaciones crece de forma explosiva: con 10 ciudades ya son 3.62×10^6 (10!). López, Salas y Murillo (2014) también definen el problema con una matriz de distancias de n × n y representan un recorrido con una permutación cíclica, en la que cada posición indica qué ciudad sigue a otra; el costo del recorrido es la suma de esas distancias.

2.4.2. Representación

Datos del ejemplo. Cuatro ciudades, A, B, C y D, ubicadas en las esquinas de un rectángulo de 4 × 3: A en (0, 0), B en (4, 0), C en (4, 3) y D en (0, 3). Las distancias en línea recta se ponen en una matriz de distancias, donde el número de la fila X y la columna Y es la distancia entre X e Y:

	A	B	C	D
A	0	4	5	3
B	4	0	3	5
C	5	3	0	4
D	3	5	4	0

Una solución es una lista con el orden en que se visitan las ciudades. Se entiende que después de la última ciudad se regresa a la primera. Por ejemplo, la lista A, B, C, D significa: salir de A, ir a B, luego a C, luego a D y regresar a A. En la computadora las ciudades se numeran (A = 0, B = 1, C = 2, D = 3), y la misma lista se guarda como 0, 1, 2, 3. López, Salas y Murillo (2014) usan otra forma equivalente: indicar, para cada ciudad, cuál es la siguiente; el mismo recorrido se escribe B, C, D, A (después de A va B, después de B va C, después de C va D y después de D regresa A).

La lista tiene exactamente una vez cada ciudad, así que cualquier reordenamiento de la lista es un recorrido válido: no se pueden omitir ni repetir ciudades.

Función de costo: la suma de las distancias entre ciudades consecutivas de la lista, más la distancia de la última a la primera. Estos son los 3 recorridos distintos del ejemplo:

Lista	Tramos	Distancia total
A, B, C, D	A–B (4) + B–C (3) + C–D (4) + D–A (3)	**14**
A, B, D, C	A–B (4) + B–D (5) + D–C (4) + C–A (5)	18
A, C, B, D	A–C (5) + C–B (3) + B–D (5) + D–A (3)	16

El mejor recorrido es A, B, C, D, que recorre el borde del rectángulo y mide 14. Los otros dos tienen cruces: pasan por las diagonales.

Una observación: la misma ruta se puede escribir de varias formas. A, B, C, D; B, C, D, A; y D, C, B, A describen el mismo recorrido, porque da igual en qué ciudad se empiece y en qué sentido se recorra. Con 4 ciudades hay 4 × 2 = 8 listas para cada recorrido, y 4! = 24 listas en total, que corresponden a 24 / 8 = 3 recorridos distintos.

Cómo se puede modificar una solución. Un cambio sencillo es invertir un tramo del recorrido, llamado movimiento 2-opt. Sea el recorrido 0, 1, 2, 3, 4, 5. Si se invierte el tramo entre las posiciones 2 y 4, queda 0, 1, 4, 3, 2, 5. En el recorrido se quitan las aristas 1–2 y 4–5 y se ponen 1–4 y 2–5; el resto sigue igual, sólo que el tramo del medio se recorre al revés. Con n ciudades hay n(n − 1)/2 cambios de este tipo. Si las dos aristas nuevas suman menos que las dos quitadas, el recorrido mejoró.

2.4.3. Solución propuesta: recocido simulado

Para el TSP se propone el recocido simulado. Parte de un recorrido y prueba cambios pequeños al azar; si el recorrido queda más corto lo acepta siempre, y si queda más largo lo acepta sólo a veces, con una probabilidad que baja a medida que baja la "temperatura" (Hernández Constantino, 2019). Se eligió por estas razones:

- **El vecindario crece rápido.** Con n ciudades hay n(n − 1)/2 inversiones posibles: 28 con 8 ciudades, 435 con 30 y 4 950 con 100. La búsqueda tabú tiene que calcular la longitud de todos esos vecinos en cada iteración, y esa revisión es el paso más costoso de ese método (Hernández Constantino, 2019). El recocido simulado prueba un solo vecino por paso.
- **Aceptar recorridos peores ayuda a deshacer cruces.** A veces para quitar un cruce hay que pasar primero por un recorrido más largo; con temperatura alta eso se permite.
- **La búsqueda tabú también sirve** para este problema (López et al., 2014), pero para recorridos grandes cada iteración sale más cara.

Cómo funciona, paso a paso:

- **Solución inicial:** un orden de ciudades al azar.
- **Costo:** la distancia total del recorrido cerrado.

- **Vecino:** se eligen dos posiciones al azar y se invierte el tramo que hay entre ellas.
- **Aceptación:** si el recorrido se acorta, se acepta. Si se alarga en Δ, se acepta con probabilidad $e^{-\Delta/T}$.
- **Enfriamiento:** tras varios intentos con la misma temperatura, T se multiplica por un factor menor que 1 (por ejemplo 0.95).
- **Fin:** cuando T llega a un valor mínimo. Siempre se guarda el mejor recorrido visto.

Ejemplo con las 4 ciudades. Si el recorrido actual es A, B, D, C (distancia 18) y se invierte el tramo de las dos últimas ciudades, resulta A, B, C, D (distancia 14): mejora, así que se acepta siempre. En cambio, si el recorrido actual es el mejor, A, B, C, D (14), y se invierte el tramo B, C, resulta A, C, B, D (16): empeora en Δ = 2, y la decisión depende de la temperatura:

Temperatura T	Probabilidad $e^{-2/T}$	Qué hace el algoritmo
10	0.82	Acepta casi siempre: explora
1	0.14	Acepta pocas veces
0.1	0.000000002	Casi nunca: sólo acepta mejoras

Valores de los parámetros. La temperatura inicial debe ser grande frente a los cambios de distancia típicos, para que al principio se acepten casi todos los empeoramientos. El factor de enfriamiento suele estar entre 0.90 y 0.99: mientras más cerca de 1, enfría más lento y explora más, pero tarda más. Para las 4 u 8 ciudades se sugiere T inicial = 5, factor 0.95, temperatura mínima 0.01 y 20 intentos por temperatura. Elegir cómo baja la temperatura es la principal dificultad del método (Hernández Constantino, 2019).

3. Conclusiones

Los cuatro problemas se ven distintos (un taller, un tablero, una red de cables y un mapa de ciudades), pero se trabajan con los mismos pasos: entender las reglas y el objetivo, escribir una solución de forma que la computadora pueda guardarla, medir su calidad con una función de costo y definir cambios pequeños que lleven de una solución a otra.

La representación fue la decisión más importante en cada caso. En el JSSP, la lista de trabajos con repeticiones hace que cualquier orden sea válido, mientras que escribir el orden de cada máquina permite contradicciones (Hernández Constantino, 2019). En las N reinas, guardar la columna de cada fila elimina por construcción los ataques por fila y por columna. En el TSP, una lista con el orden de las ciudades siempre es un recorrido válido (López et al., 2014). En el MST no se logró una representación que evite todas las soluciones inválidas, y se resolvió con una penalización en el costo.

También se vio que la metaheurística conveniente depende del problema. Se propuso la búsqueda tabú cuando el vecindario es pequeño y revisarlo completo es barato (MST) o

cuando la literatura la señala como la mejor opción (JSSP), y el recocido simulado cuando el vecindario crece rápido con el tamaño del problema y conviene probar un solo vecino por paso (N reinas y TSP).

Problema	Cómo se representa	Algoritmo propuesto	Razón principal
JSSP	Lista de números de trabajo con repeticiones	Búsqueda tabú	Es la que mejores resultados ha dado para este problema (Hernández Constantino, 2019)
N reinas	Lista con la columna de la reina de cada fila	Recocido simulado	El vecindario crece con N; prueba un vecino por paso y es muy eficiente (Alba y Durán, 1997)
MST	Lista de IDs de las aristas elegidas	Búsqueda tabú	Vecindario pequeño (20 cambios): revisarlo completo es barato
TSP	Lista con el orden de las ciudades	Recocido simulado	El vecindario crece como $n(n-1)/2$; prueba un vecino por paso

Este trabajo se centró en entender y representar los problemas con instancias pequeñas, por lo que no se midió el desempeño de los algoritmos. Para compararlos en serio habría que programarlos y probarlos con instancias de referencia y varias ejecuciones. Lo que sí queda claro es que entender el problema, su representación y su costo es lo que permite elegir y ajustar el algoritmo; sin eso, cualquier método de búsqueda funcionaría a ciegas.

4. Bibliografía

ALBA, E., & DURÁN, J. C. (1997). *Un acercamiento natural al problema de las N-Reinas*. En V. Botti (Coord.), CAEPIA'97: actas (pp. 725–734). Asociación Española para la Inteligencia Artificial. Obtenido de https://portaldelainvestigacion.uma.es/documentos/6631363538b3d85bb1447876

DÍAZ LEYVA, P. A., & MEDERO MARTÍNEZ, M. (agosto de 2020). *Modelación de la programación (scheduling) de la producción con tiempos de transporte, ajuste y configuración de máquinas en talleres de mecanizado convencional*. Revista de Desarrollo Sustentable, Negocios, Emprendimiento y Educación (RILCO DS)(10). Obtenido de https://www.eumed.net/rev/rilcoDS/10/programacion-talleres.html

DIAZ RODRIGUEZ, B. I., GONZALES MONDRAGON, R. L., PEREZ GONZALEZ, A. K., RAMIREZ QUISPE, E. J., & SANCHEZ SANGAMA, M. A. (2026). *Algoritmo de Prim para la obtención de árboles de expansión mínima en grafos ponderados*. Revista Científica TechNoLogica, 1(2), 8–14. Obtenido de https://doi.org/10.17268/tech1.2.2026.02

HERNÁNDEZ CONSTANTINO, O. (10 de diciembre de 2019). *Metaheurísticas aplicadas al problema de planificación para producción tipo taller* [Tesis de maestría]. CIMAT. Obtenido de https://cimat.repositorioinstitucional.mx/jspui/handle/1008/1037

LÓPEZ, E., SALAS, Ó., & MURILLO, Á. (2014). *El problema del agente viajero: un algoritmo determinístico usando búsqueda tabú*. Revista de Matemática: Teoría y Aplicaciones, 21(1), 127–144. Obtenido de http://www.redalyc.org/articulo.oa?id=45331281008

ORTEGA, M., & RAMOS, M. (2012). *Evaluación de la eficiencia del algoritmo de Kruskal para la construcción de árboles cobertores mínimos*. Prisma Tecnológico, 3(1), 57–60. Obtenido de https://doaj.org/article/b3c302f266b3478c9633053ef17c43b1

OSABA ICEDO, E. (2020). *Diseño e implementación de una meta-heurística multi-poblacional de optimización combinatoria enfocada a la resolución de problemas de asignación de rutas a vehículos* [Tesis doctoral]. Bilbao. Obtenido de https://arxiv.org/pdf/2003.11393